

Практическая работа 10 по Информационным технологиям

Системные вызовы RARS

Для выполнения этой практической работы вам потребуются материалы [лекции 10 «Системные вызовы»](#).

В таблице 1 приведены некоторые системные вызовы из справки RARS.

Таблица 1. Некоторые системные вызовы RARS

Функция	Тип a7	Параметры	Возвращаемое значение
вывод целого	1	a0 = целое	
вывод вещественного	2	fa0 = вещественное	
вывод вещественного двойной длины	3	fa0 = двойное вещественное	
вывод строки	4	a0 = адрес ASCIIZ- строки	
ввод целого	5		a0 — введённое целое (word)
ввод вещественного	6		fa0 — введённое вещественное (float)
ввод двойного вещественного	7		fa0 — введенное двойное вещественное (double)
ввод строки	8	a0 = адрес памяти для размещения строки a1 = размер буфера	Если строка короче буфера, все её символы записываются в буфер (включая '\n', если он был), после чего туда дописывается нулевой байт. Если строка не короче буфера, в него записывается первые a1-1 символов, а в последний оставшийся байт записывается 0 Таким образом формируется ASCIIZ-строка
выделение памяти в Куче (sbrk)	9	a0 = размер требуемой памяти в байтах	a0 — адрес требуемой памяти
останов (exit)	10		
вывод символа	11	a0 = символ	Некоторые управляющие символы, например, перевод строки (код 10) тоже имеет смысл выводить
ввод символа	12		a0 — считанный символ (традиционно в RARS работает отвратительно, не рекомендуется 😞)
системное время (time)	30		Время выдаётся в миллисекундах с начала мира (полночь на 1 января 1970 года), так что помещается как длинное целое в два слова a1 = старшее слово a0 = младшее слово
приостановка выполнения (sleep)	32	a0 = время простоя в миллисекундах.	
вывод шестнадцатеричного	34	a0 = целое	
вывод двоичного	35	a0 = целое	
вывод беззнакового	36	a0 = целое	

Для решения задач надо использовать связку «макрос+подпрограмма» — чтобы убедиться в том, насколько структуризация и повторное использование делает исходные тексты понятнее.

Первое задание делают все, задания со 2 по 5 – по вариантам.

Задание 1. «Строки» (StrMacro)

Написать комплект «макрос+подпрограмма» для функция работы со строками. Формат вызова, точный список функций и логика их работы **на ваше усмотрение**.

- Пример реализации – в таблице ниже (все параметры — это регистры, которые перед вызовом подпрограммы копируются в соответствующие **a***-регистры).
-  В этом примере есть макросы, имена которых совпадают с именами в прошлом задании, а вот работа несколько отличается.

Макрокоманда	Параметр 1	Параметр 2	Описание	→a0	→a1
<code>strlen</code>	Адрес		Определение длины ASCIIZ-строки	Длина строки	
<code>strget</code>			Ввод ASCIIZ-строки длиной не более 100 байтов. Ввод происходит в статический буфер, далее для строки выделяется соответствующее место на куче, и она копируется туда (<code>\n</code> , если есть, из строки удаляется, в конец обязательно записывается 0)	Адрес выделенной памяти	Длина строки
<code>strcpy</code>	Приёмник	Источник	(Небезопасно) копирует строку с адреса «источник» по адресу «приёмник»	Приёмник	Длина строки-приёмника
<code>strcmp</code>	Адрес 1	Адрес 2	Лексикографическое сравнение ASCIIZ-строк. Возвращает отрицательное число, если первая строка меньше, положительное, если больше и ноль, если строки равны.	Результат	
<code>strchr</code>	Адрес 1	Байт	Возвращает адрес первого вхождения символа <code>char</code> в строку по адресу <code>addr1</code> или 0, если символ в строке отсутствует	Адрес 2	

- В вызове `esall` 8 параметр **a1** — это именно размер буфера, включая `\0` в его конце. Символов вводится при этом на 1 меньше.

- Ещё один вариант — отказаться от использования **a***-регистров в тексте программы и предусмотреть в макросах копирование результатов оттуда. Например, `strlen %регистр_адреса %регистр_длины` будет копировать `%регистр_адреса` в **a0**, вызывать внутреннюю подпрограмму, реализующую `strlen`, получать в **a0** длину и записывать её в `%регистр_длины`.
- В части с макросами можно понаписать всякого удобного: отладочную выдачу, `push/rop`, `ecall` одной строкой (причём с разным количеством параметров!) и, разумеется, *пролог* и *эпилог* подпрограммы.
- Макросы, библиотеку подпрограмм и основной текст `main` (не забываем про `.globl main`) удобно хранить в разных файлах и использовать многофайловую сборку.
- 🟡 В решениях разрешается хранить в памяти все ведённые с помощью `strget` строки.
- 🟡 В решениях разрешается рассчитывать на то, что `ecall 9` выделяет память на куче строго подряд (с учётом выравнивания по слову).

Задание 2. «Пирамидка» (LongerLines)

Вводить строки, чья длина не превышает 100 байтов (без учёта перевода строки, который может быть 101-м), выводить только те из них, что не короче предыдущей выведенной строки. Ввод заканчивается пустой строкой.

Input:

A foolish consistency is the hobgoblin of little minds,
 adored by little statesmen and philosophers and divines.
 With consistency a great soul has simply nothing to do.
 He may as well concern himself with his shadow on the wall.
 Speak what you think now in hard words, and to-morrow speak
 what to-morrow thinks in hard words again,
 though it contradict every thing you said to-day.
 — 'Ah, so you shall be sure to be misunderstood.'
 — Is it so bad, then, to be misunderstood? Pythagoras was misunderstood,
 and Socrates, and Jesus, and Luther, and Copernicus, and Galileo,
 and Newton, and every pure and wise spirit that ever took flesh. To be great is to be misunderstood.

Output:

A foolish consistency is the hobgoblin of little minds,
 adored by little statesmen and philosophers and divines.
 He may as well concern himself with his shadow on the wall.
 Speak what you think now in hard words, and to-morrow speak
 — Is it so bad, then, to be misunderstood? Pythagoras was misunderstood,
 and Newton, and every pure and wise spirit that ever took flesh. To be great is to be misunderstood.

Пример реализации (без макросов и библиотеки):

```
.include "string.inc"    # сюда надо вставить string.inc и string.asm
.globl main
main:    li        s1 0
loop:    strget
        beqz      a1 fin
        blt       a1 s1 loop
        mv        s1 a1
        syscall 4
        putchar '\n'
```

```
j      loop
fin:   syscall 10
```

Задание 3. «Подстрока» (StrGrep)

Ввести непустую строку-шаблон. Затем вводить строки (ввод заканчивается пустой строкой) и выводить только те из них, что содержат строку-шаблон.

Input:

```
given
Laws of Computer Programming:
(1) Any given program, when running, is obsolete.
(2) Any given program costs more and takes longer.
(3) If a program is useful, it will have to be changed.
(4) If a program is useless, it will have to be documented.
(5) Any given program will expand to fill all available memory.
(6) The value of a program is proportional to the weight of its output.
(7) Program complexity grows until it exceeds the capability of the
programmer who must maintain it.
```

Output:

```
(1) Any given program, when running, is obsolete.
(2) Any given program costs more and takes longer.
(5) Any given program will expand to fill all available memory.
```

Задание 4. «Повторение последней» (SameLast)

Ввести несколько строк длины не более 100 символов (ввод заканчивается пустой строкой). Посчитать, сколько раз среди них встречается последняя введенная непустая строка. Допускается рассчитывать на то, что `ecall 9` выделяет память на куче строго подряд с учётом выравнивания.

Input:

```
There was a man lived in the moon, lived in the moon, lived in the moon,
There was a man lived in the moon,
And his name was Aiken Drum.
And he played upon a ladle, a ladle, a ladle,
And he played upon a ladle,
And his name was Aiken Drum.
And his hat was made of good cream cheese, of good cream cheese, of good
cream cheese,
And his hat was made of good cream cheese,
And his name was Aiken Drum.
And his coat was made of good roast beef, of good roast beef, of good roast
beef,
And his coat was made of good roast beef,
And his name was Aiken Drum.
```

Output:

Задание 5. «Сортировка множества» (UinqSort)

Ввести число N, а затем — N строк, вывести их в отсортированном лексикографически порядке, опуская повторы.

Input:

```
10
A-tisket a-tasket
A green and yellow basket
I wrote a letter to my friend
And on the way I dropped it,
I dropped it, I dropped it,
And on the way I dropped it,
A little boy he picked it up
And put it in his pocket.
A-tisket a-tasket
A green and yellow basket
```

Output:

```
A green and yellow basket
A little boy he picked it up
A-tisket a-tasket
And on the way I dropped it,
And put it in his pocket.
I dropped it, I dropped it,
I wrote a letter to my friend
```

Контрольные вопросы

1. Что такое *ecall*?
2. Что такое системный вызов?
3. Кто выполняет системный вызов?
4. Куда происходит возврат из системного вызова?